



Codenix — Assistant IA conversationnel

Rapport technique complet du projet

Version 1.0 — Septembre 2026 — Rédigé en français
Chat IA (Groq) + Q&A sur PDF (RAG) + historique persistant + CI/CD

Sommaire

1. Présentation générale
2. Technologies utilisées
3. Architecture globale
4. Architecture détaillée du backend
5. Architecture détaillée du frontend
6. Documentation API (Swagger / OpenAPI)
7. Diagramme de cas d'utilisation
8. Diagramme de classes (domaine)
9. Diagrammes de séquence
10. Base de données (MongoDB + FAISS)
11. DevOps (CI/CD, qualité, charge)
12. Déploiement
13. Stratégie de versions et montée de version
14. Technologies à adopter
15. Roadmap des fonctionnalités + Conclusion
16. RAG approfondi (entretien)
17. LLM & Groq approfondis (entretien)
18. Fiche d'entretien express (Q/R)

1. Présentation générale

Codenix est une application web full-stack : frontend **Next.js**, backend **FastAPI**. Un utilisateur authentifié (Clerk) dialogue avec un modèle d'IA (Groq), peut joindre un **PDF à son message** pour des réponses fondées sur le document (RAG), et retrouve son **historique** persisté.

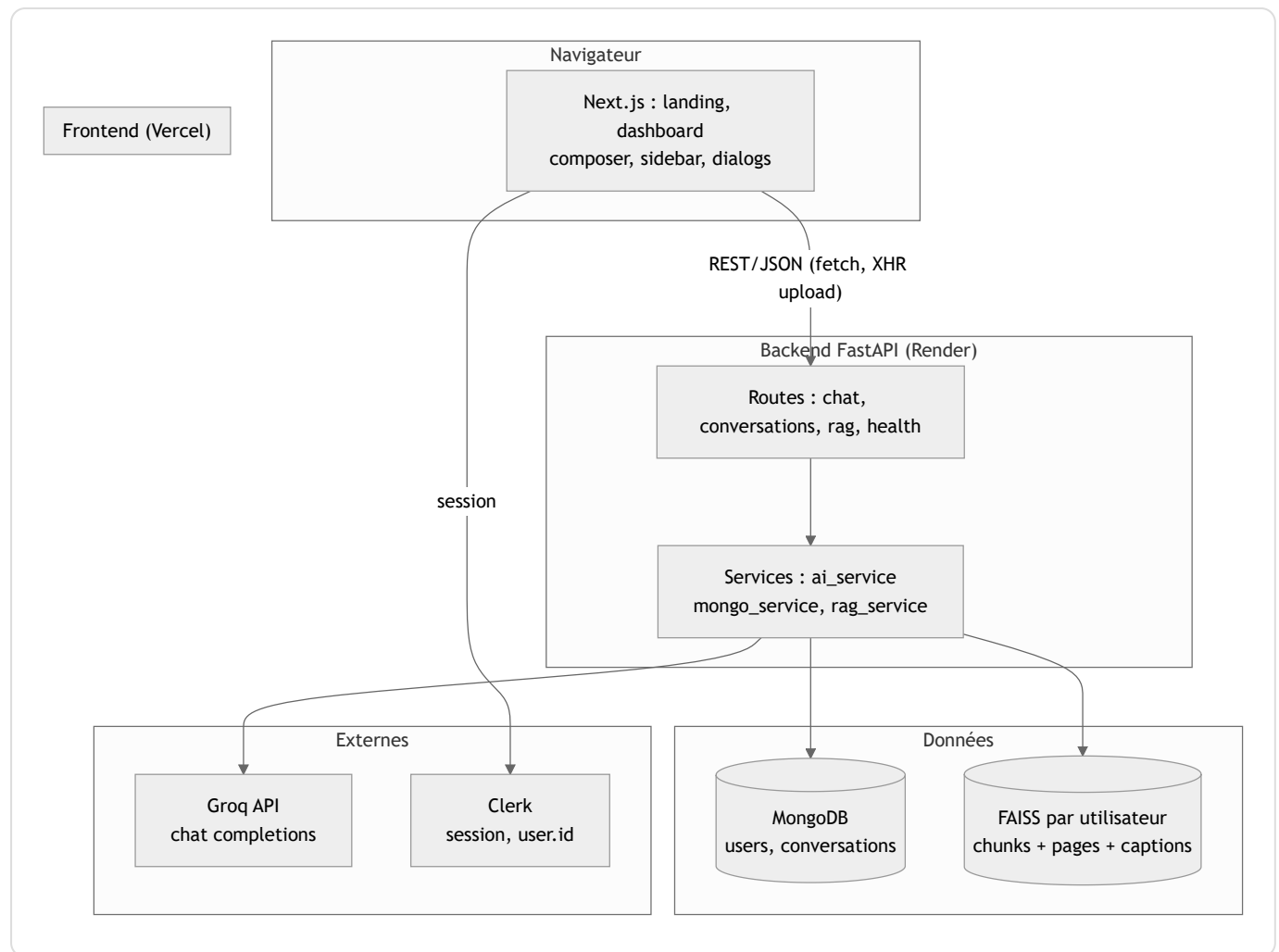
Fonctionnalités principales

- Chat temps réel (typewriter), Markdown riche, coloration syntaxique + copie.
- Envoi combiné *PDF* + *message* avec progression 0–100 %.
- Modes **Ask / Agent**, sélecteur de modèle, palette Ctrl+K.
- Thèmes sombre/clair, sidebar redimensionnable, suppression avec confirmation.

2. Technologies utilisées

Couche	Technologie	Version	Rôle
Frontend	Next.js (App Router)	15.5.26	SSR/SSG, routage, groupes de routes
Frontend	React + TypeScript	19 / 5	UI, typage strict
Frontend	Tailwind CSS	4 (tokens oklch)	Thème sombre/clair
Frontend	Clerk	6.24	Auth (groupe dashboard uniquement)
Frontend	PrismLight	15.6.1	Code (14 langages)
Backend	FastAPI + Uvicorn	0.141.1 / 0.54.0	API REST asynchrone + Swagger
Backend	Pydantic	2.13.5	Validation (400/422)
IA	Groq (AsyncGroq)	1.7.0	Completions + fallback multi-modèles
Base	MongoDB (PyMongo)	4.18.2 / 7	Users + conversations (Atlas ou Docker)
RAG	FAISS + MiniLM + LangChain	1.15.1 / 6.1.0 / 1.4.2	Index vectoriel par utilisateur
PDF	pypdf	6.19.0	Texte, pages, figures, captions
Tests	pytest / Vitest / Playwright / Locust	9.1.1 / 4 / 1.63	30 + 26 + 6 tests, charge 10k
DevOps	GitHub Actions, SonarQube, Docker, Dependabot	—	CI, qualité, versions
Déploiement	Vercel (front) + Render (back)	—	Serverless + serveur persistant

3. Architecture globale



4. Architecture détaillée du backend

Client (navigateur)

Middleware
CORS + GZip + lifespan
(index)

api/routes/
chat · conversations · rag ·
health

schemas/ (Pydantic)
ChatRequest,
ConversationIn...
→ 400/422 automatiques

services/
ai_service : AsyncGroq +
fallback + sanitize
mongo_service : pool 50,
ping 30s, index
rag_service : FAISS + cache
mtime

Groq · MongoDB · FAISS
disque

`main.py` ne contient que la fabrique (CORS, GZip, lifespan). Les routes sont fines, les schémas valident, les services portent le métier. Erreurs mappées : 503 (dépendances), 502 (provider avec cause), 500 (filet).

Arborescence du dossier backend

```
backend/
├─ main.py                # entrée uvicorn : uvicorn main:app
├─ Dockerfile             # image prod (Render / Docker)
├─ requirements.txt        # dépendances applicatives
├─ requirements-test.txt   # pytest, pytest-asyncio, httpx
├─ requirements-loadtest.txt # locust (charge 10k users)
├─ pytest.ini             # config tests (asyncio auto)
├─ .env.example           # modèle des variables d'environnement
├─ loadtest/locustfile.py  # scénario de charge (tags ai / infra)
├─ tests/                 # 30 tests : test_api, test_ai_service, test_rag_service
├─ app/
│   ├─ main.py            # fabrique : CORS, GZip, lifespan, Swagger/OpenAPI
│   ├─ core/              # config.py (env centralisé), logging.py
│   ├─ schemas/           # modèles Pydantic : chat, conversations, rag
│   ├─ api/
│   │   ├─ deps.py        # décorateur : erreurs métier → codes HTTP
│   │   └─ routes/        # chat.py, conversations.py, rag.py, health.py
│   └─ services/
│       ├─ ai_service.py   # AsyncGroq, fallback multi-modèles, sanitize
│       ├─ mongo_service.py # client paresseux, pool 50, index au démarrage
│       └─ rag_service.py  # ingestion PDF, FAISS par user + cache mtime
```

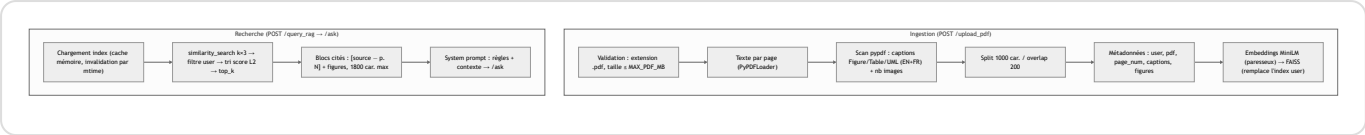
Configuration Groq en détail

Variable	Défaut	Rôle
GROQ_API_KEY	— (obligatoire)	Clé API. Jamais loggée ; absente → 503 explicite au lieu d'un crash au boot (client paresseux).
GROQ_MODEL	openai/gpt-oss-20b	Modèle principal (vérifié accessible). Modifiable par requête (allow-list).
GROQ_FALLBACK_MODELS	Llama 3.1 8B, Llama 4 Scout, Qwen3 32B, GPT-OSS 120B	Chaîne de secours sur <code>model_not_found</code> uniquement.
GROQ_TIMEOUT_SECONDS	60	Timeout HTTP du client <code>AsyncGroq</code> (singleton partagé).
GROQ_MAX_TOKENS	4096	Les modèles de raisonnement consomment des tokens en réfléchissant ; 1024 tronquait les longues réponses.

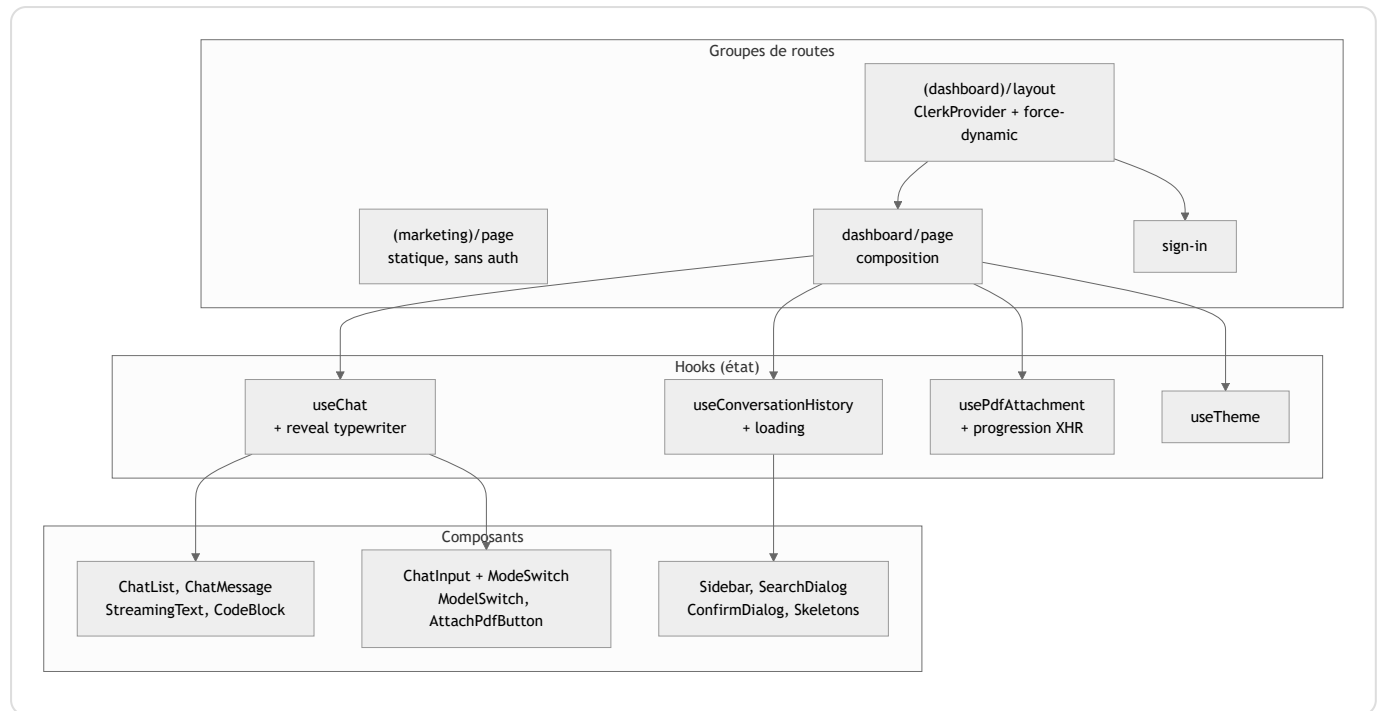
Chaîne d'appel : modèle demandé → `GROQ_MODEL` → fallbacks (dédupliqués, ordre gardé). Seule l'erreur `model_not_found` déclenche le suivant ; 401/429/timeout remontent aussitôt avec un message clair (502

+ detail). Le modèle effectivement utilisé est retourné ({response, model}) et affiché dans le header.
GET /api/models expose la allow-list (id + label) ; tout autre id → 400.

Pipeline RAG en détail



5. Architecture détaillée du frontend



Client API typé (`lib/api.ts` , `ApiError` avec le `detail` backend), helpers purs testés (`lib/messages.ts`), classes sémantiques de thème (aucune couleur en dur).

Arborescence du dossier frontend

```

frontend/
├─ app/
│   ├─ layout.tsx          # racine : fonts, boot thème, metadata (SANS auth)
│   ├─ icon.svg            # favicon Codenix (servi auto par Next.js)
│   ├─ globals.css         # tokens oklch :root/.dark + classes sémantiques
│   ├─ (marketing)/page.tsx # landing 100% statique (zéro JS d'auth)
│   └─ (dashboard)/
│       ├─ layout.tsx      # frontière auth + force-dynamic (jamais pré-rendu)
│       ├─ dashboard/
│       └─ sign-in/        # <SignIn/> Clerk thémé via appearance
├─ components/chat/        # 13 composants : Header, Sidebar, Chatlist,
│                           # ChatMessage, StreamingText, CodeBlock, ChatInput,
│                           # ModeSwitch, ModelSwitch, AttachPdfButton,
│                           # SearchDialog, ConfirmDialog, ChatSkeleton
├─ components/layout/      # Logo (bulle + éclair), ThemeToggle, VersionBadge
├─ hooks/                  # useChat, useConversationHistory,
│                           # usePdfAttachment, useTheme (tous testés/stables)
├─ lib/                    # config.ts (APP_VERSION, API URL),
│                           # api.ts (client typé + upload XHR),
│                           # types.ts (ChatMessage, ChatMode, ModelOption),
│                           # messages.ts (payload, prompts, titres)
├─ layouts/                # Header/Footer statiques (landing, sans Clerk)
├─ tests/ + e2e/           # 26 tests Vitest + 6 specs Playwright
└─ scripts/export-report-pdf.mjs # génère ce PDF (npm run report:pdf)

```

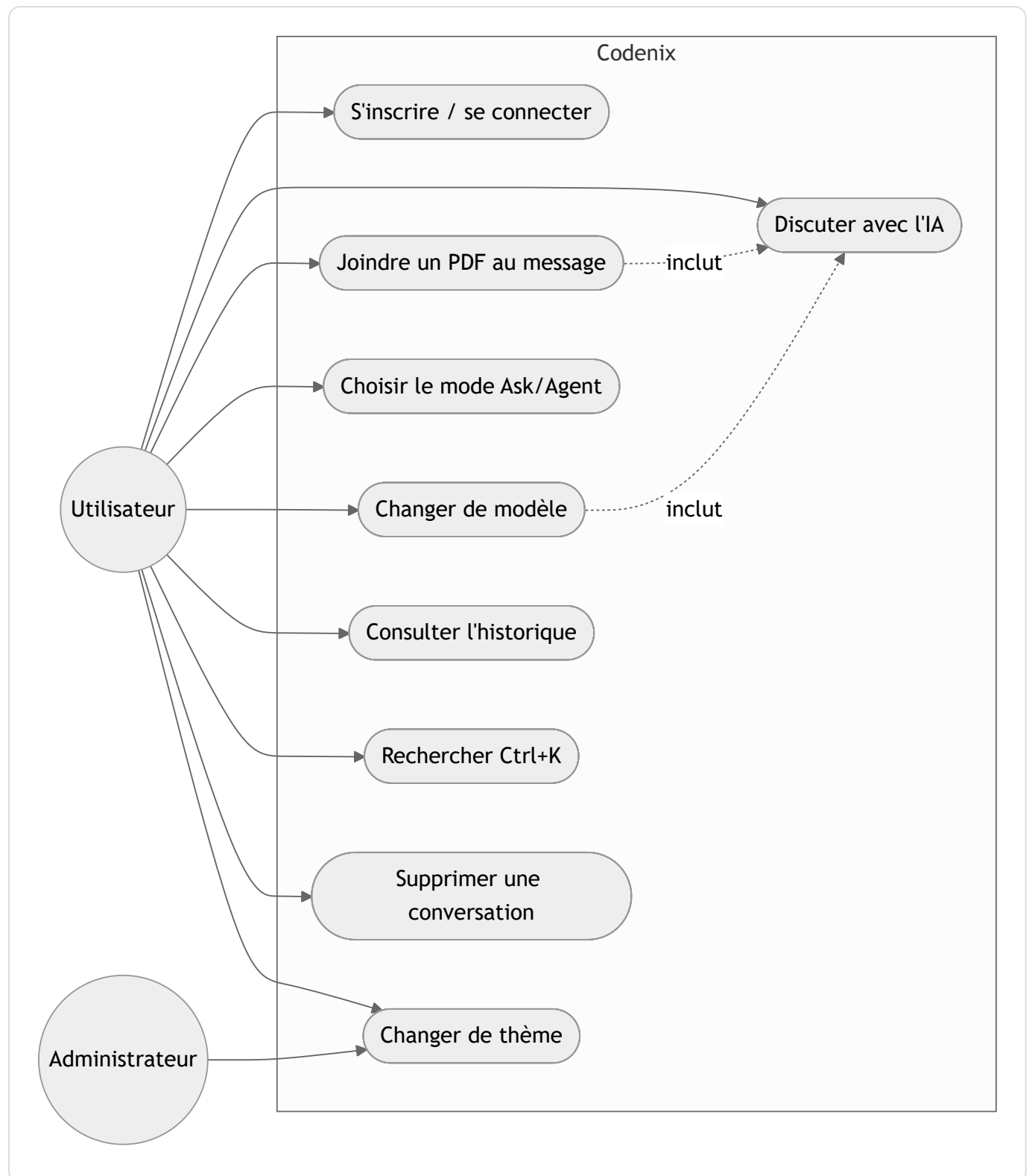
6. Documentation API (Swagger / OpenAPI)

FastAPI expose nativement une documentation interactive, enrichie (titre *Codenix API v1.0.0*, tags par domaine) :

- `GET /docs` — Swagger UI : essayer chaque endpoint depuis le navigateur.
- `GET /redoc` — variante ReDoc, lecture seule.
- `GET /openapi.json` — schéma machine (11 paths) pour générer des clients.

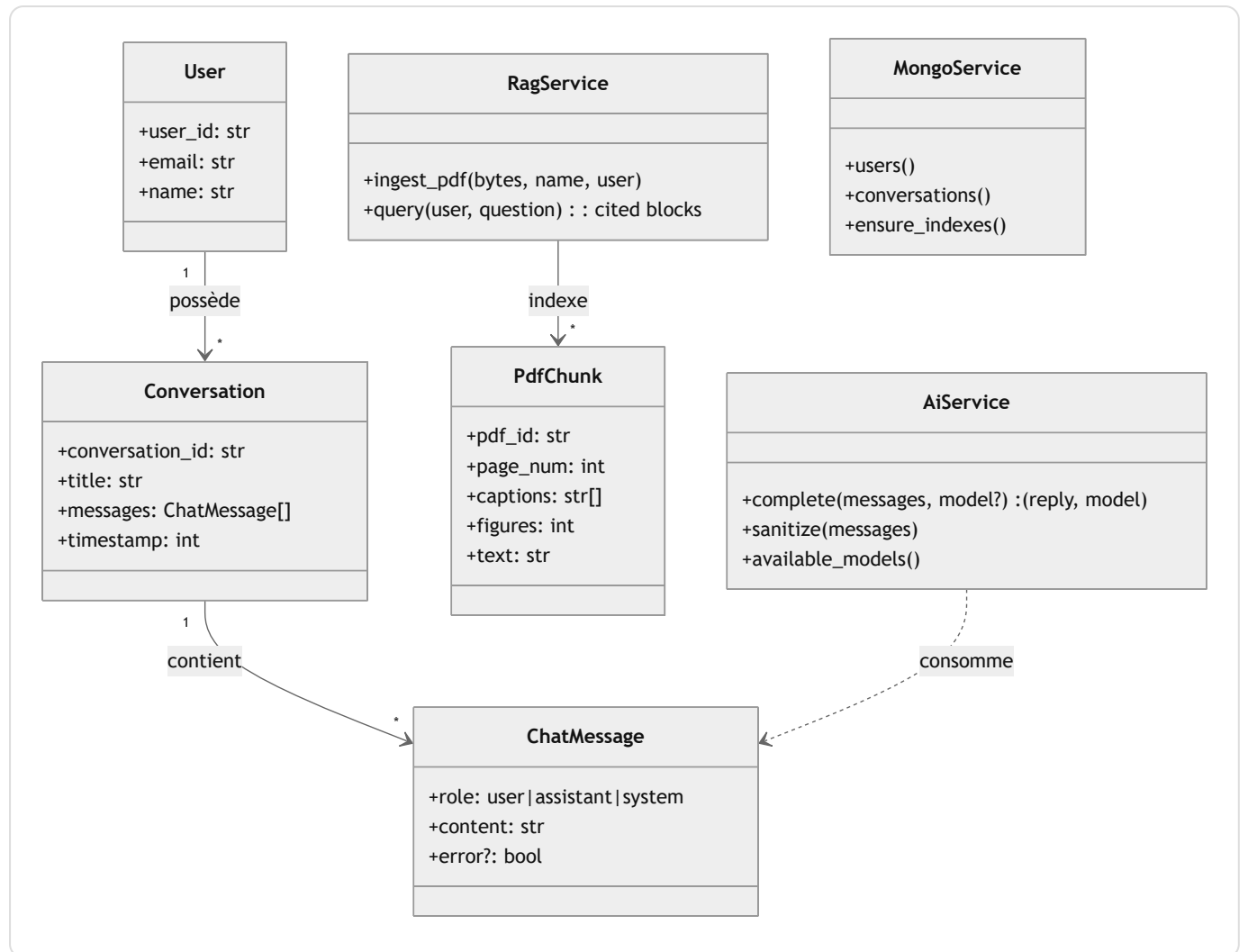
Tag	Endpoints couverts
chat	<code>/ask</code> , <code>/api/models</code>
conversations	<code>store_user</code> , <code>save/get/delete_conversation</code>
rag	<code>upload_pdf</code> , <code>query_rag</code> , <code>list_chunks</code> , <code>vector_health</code>
health	<code>/health</code>

7. Diagramme de cas d'utilisation



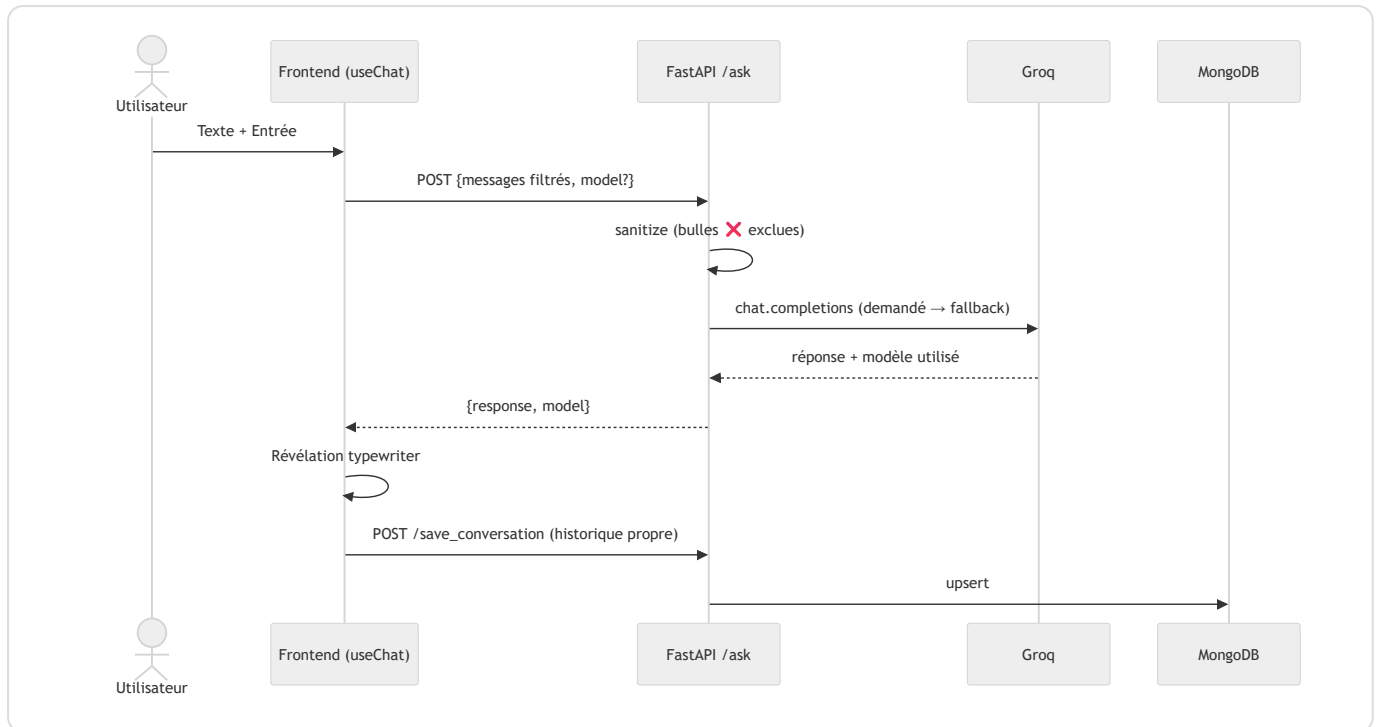
Acteur	Cas	Description
Utilisateur	Discuter	Message → réponse animée, formatée, persistée
Utilisateur	PDF + message	Chip d'attente, upload 0–100 %, réponse fondée sur le document
Utilisateur	Modes / modèle	Style Ask ou Agent, modèle commutable avec fallback
Utilisateur	Historique	Liste, recherche, suppression confirmée
Administrateur	Superviser	/health, logs, Sonar, Locust, versions

8. Diagramme de classes (domaine)

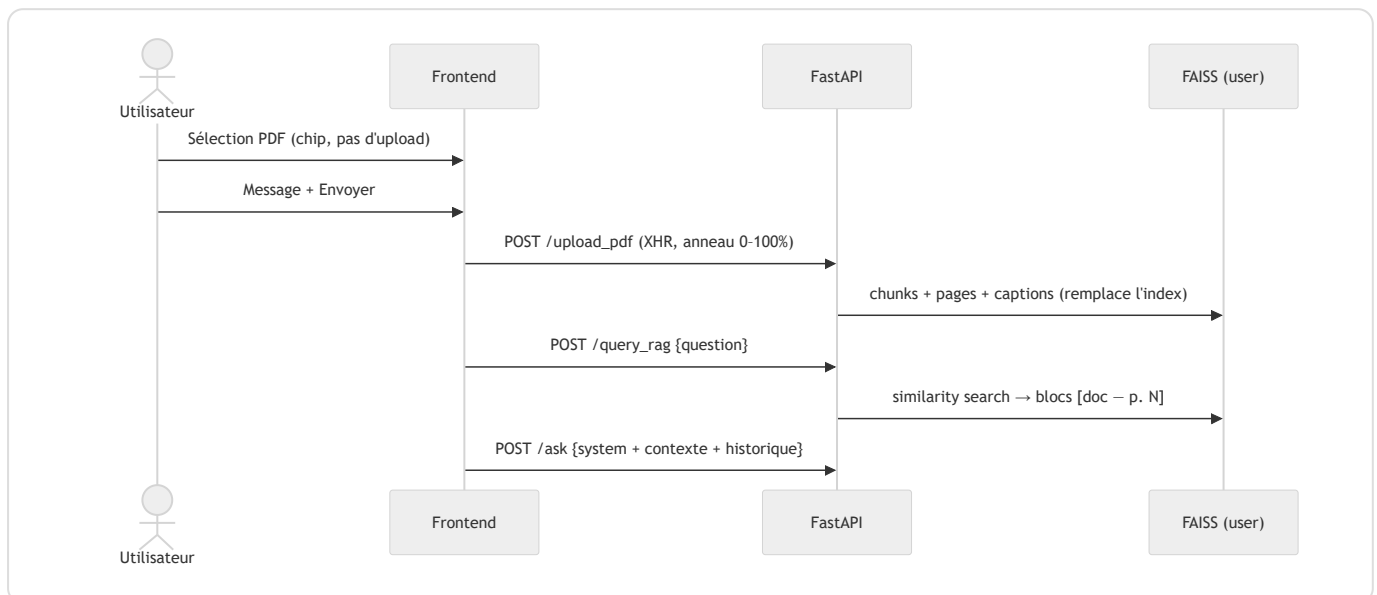


9. Diagrammes de séquence

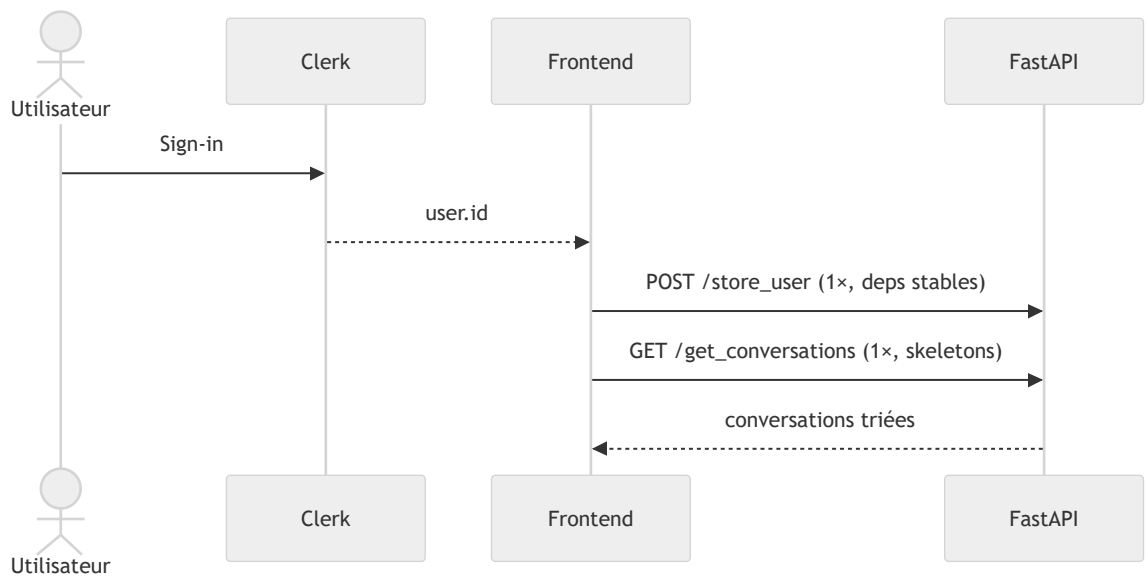
9.1 Envoi d'un message simple



9.2 Message avec PDF (RAG)



9.3 Connexion et historique

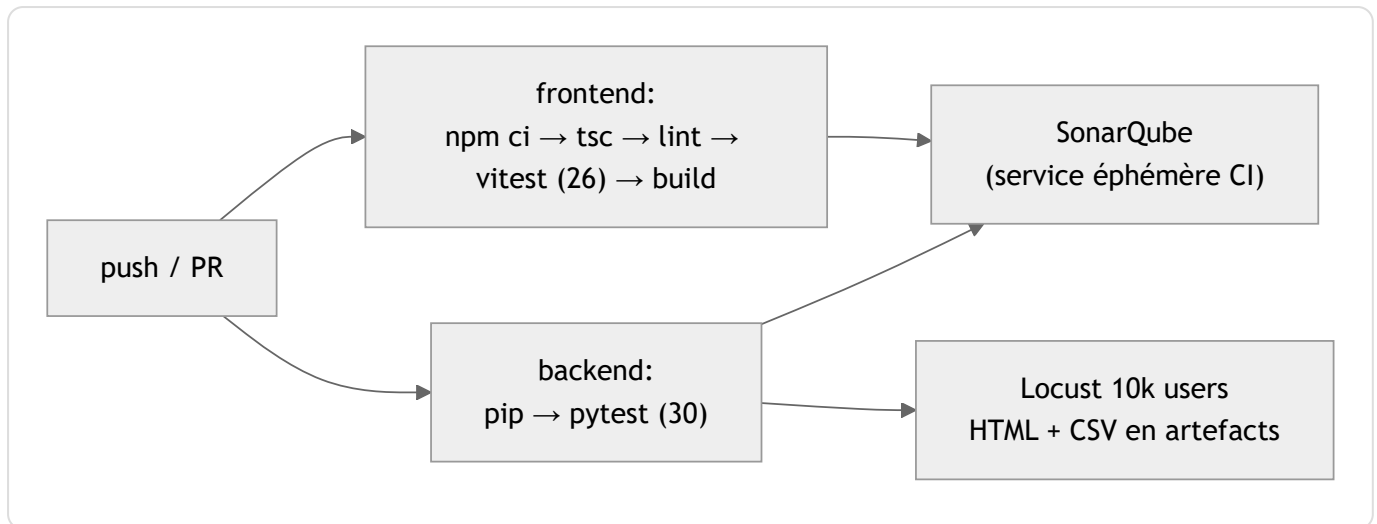


10. Base de données (MongoDB + FAISS)

Collection	Champs	Index
users	user_id, email, name	user_id unique
conversations	user_id, conversation_id, title, messages[], timestamp	(user_id, timestamp ↓)

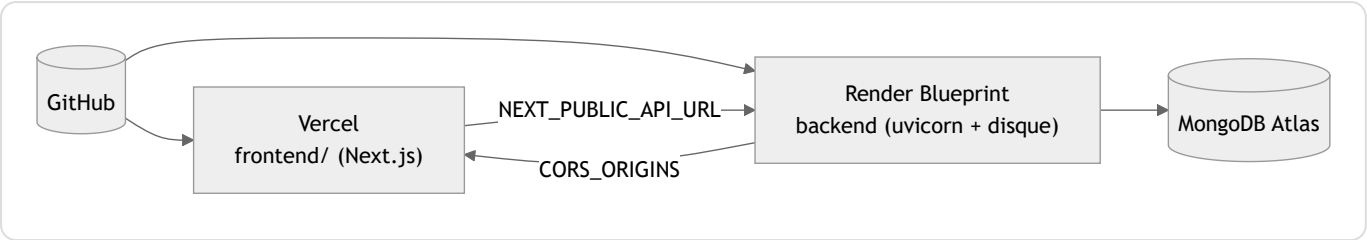
FAISS sur disque, isolé par utilisateur, cache mémoire invalidé par mtime. MongoDB : Atlas ou conteneur Docker (`docker compose up`).

11. DevOps (CI/CD, qualité, charge)



- **CI GitHub Actions + Dependabot** hebdomadaire (npm + pip, CI verte exigée).
- **Tests** : 30 pytest (mockés), 26 Vitest, 6 Playwright, Locust tags `ai / infra`.
- **Contraintes apprises** : plafond Windows `select()` 512 FD (workers + limit-concurrency), quota Groq (runs `--exclude-tags ai`).

12. Déploiement



Cible	Procédure	Variables
Backend (Render)	New → Blueprint (<code>render.yaml</code> : <code>build</code> , <code>\${PORT:-8000}</code> , <code>disque /var/data</code> , <code>/health</code>)	GROQ_API_KEY, MONGO_URI, CORS_ORIGINS, FAISS_INDEX_PATH=/var/data
Frontend (Vercel)	Import, Root Directory <code>frontend</code> , Output vide (jamais <code>public</code>), Next ≥ 15.5.26	Clés Clerk ×2, NEXT_PUBLIC_API_URL
Clerk	Domaine Vercel + redirections (/dashboard)	—
Local/Docker	<code>docker compose up --build</code> (Mongo inclus)	backend/.env

13. Stratégie de versions et montée de version

SemVer strict (MAJEURE.MINEURE.PATCH) : APP_VERSION (frontend) et version FastAPI avancent ensemble ; tout changement visible = entrée Changelog (format Keep a Changelog : Added/Changed/Fixed).

Processus de montée de version des dépendances

- 1. **Audit** : npm outdated / pip list --outdated + npm audit ; Dependabot ouvre déjà les PRs chaque lundi (npm + pip, max 5, CI exigée).
- 2. **Branche dédiée** chore/upgrade-xxx : une famille à la fois (jamais framework + libs ensemble).
- 3. **Verrous** : package-lock.json et versions requirements.txt committés ; pas de latest flottant (leçon : Next 15.3.5 → 15.5.26 épinglé).
- 4. **Validation** : tests (pytest + Vitest + Playwright), next build local, revue Swagger (/docs), déploiement staging si risque.
- 5. **Montées planifiées** :

Dépendance	Actuel → Cible	Motif
Next.js	15.5.26 → 16 (évaluer)	breaking : tester Turbopack + cache
langchain-community	0.4.2 → packages standalone	Paquet en fin de vie (warning au boot)
Node.js	20 → 22 LTS	Aligner CI + runtime
React	19 (à jour)	Surveillance mineurs
torch / sentence-transformers	Épinglés, maj trimestrielle	Éviter les régressions d'embeddings

14. Technologies à adopter (recommandées)

Technologie	Usage	Priorité
Redis	Cache des chunks RAG + rate-limit par utilisateur (protège le quota Groq)	P0
SSE (Server-Sent Events)	Streaming temps réel des réponses au lieu du typewriter simulé	P0
Sentry (front + back)	Remontée d'erreurs production avec contexte	P0
S3 / MinIO	Stockage des PDF (multi-instances, au lieu du disque local)	P1
Atlas Search / Meilisearch	Recherche plein texte + hybride (BM25 + vecteurs) dans l'historique	P1
next-intl	Internationalisation FR/AR officielle	P1
PWA (manifest + service worker)	Installation mobile, icône, mode offline partiel	P2
Prometheus + Grafana	Métriques (latence /ask, 502/429, FD) et alertes	P2
Husky + pre-commit	Lint/tests avant chaque commit (même garde-fous qu'en CI)	P2

15. Roadmap des fonctionnalités + Conclusion

Fonctionnalité	Priorité	Notes
Upload multi-PDF simultanés	P0	L'index actuel est remplacé à chaque upload
Vision LLM des figures UML	P0	Nécessite une clé avec modèle vision (Llama 4 Scout)
Partage de conversation (lien) + export PDF/Markdown	P1	Token de partage, rendu existant réutilisable
Dossiers / tags + recherche plein texte	P1	Avec Atlas Search ci-dessus
Votes 👍👎 + analytics d'usage	P1	Boucle qualité des réponses
Entrée vocale (Whisper)	P2	Groq héberge whisper-large-v3
Tableau admin (quotas, coûts)	P2	Avec Prometheus ci-dessus

Conclusion. Codenix v1.0 livre un socle complet et testé : conversation robuste, RAG cité par pages, UI moderne, CI/CD avec qualité et charge, documentation Swagger intégrée et stratégie de versions. Prochaine étape conseillée : **SSE + Redis (P0)**, puis vision des figures.

16. RAG approfondi (niveau entretien)

16.1 C'est quoi le RAG ?

Retrieval-Augmented Generation : au lieu de demander au LLM de répondre « de tête » (risque d'hallucination, ignorance de vos documents privés), on **récupère d'abord** les passages pertinents d'une base, puis on les **injecte dans le prompt**. Le modèle répond en s'appuyant sur du vrai contexte — vérifiable (citations [p. N]).

	RAG (notre choix)	Fine-tuning	Prompt seul
Données privées/récentes	✅ immédiat (upload → requêteable)	❌ ré-entraînement coûteux	❌ copier-coller manuel
Coût	Faible (index + requêtes)	Élevé (GPU, dataset)	Nul mais non scalable
Traçabilité	✅ citations de pages	❌ boîte noire	✅ (texte fourni)
Limite	Qualité du retrieval	Rigidité	Taille du contexte

16.2 Les 4 étapes dans Codenix (chiffres à connaître)

1. **Chunking** : split 1000 caractères / overlap 200. Trop gros → bruit ; trop petit → contexte perdu. L'overlap évite de couper une phrase/figure en deux.
2. **Embeddings** : all-MiniLM-L6-v2 → vecteurs de **384 dimensions**, chargés paresseusement (démarrage rapide).
3. **Recherche vectorielle** : FAISS (IndexFlatL2 , distance euclidienne — plus petit = plus proche), k×3 candidats → filtre user → tri score → top_k=4 . Index **isolé par utilisateur** + cache mémoire invalidé par mtime.
4. **Assemblage** : blocs [source – p. N] + captions de figures + plafond 1800 car., injectés en message system avec règles de réponse.

16.3 Questions d'entretien RAG (avec réponses modèles)

Question	Bonne réponse (courte)
Pourquoi RAG plutôt que fine-tuning ?	Données privées/évolutives sans ré-entraînement, coût faible, citations vérifiables. Fine-tune = style/comportement, pas connaissances.
C'est quoi un embedding ?	Vecteur dense (ici 384 dims) où la proximité géométrique $\approx$ similarité sémantique ; deux phrases synonymes ont des vecteurs proches.
Cosinus vs L2 ?	Cosinus = angle (insensible à la norme), L2 = distance euclidienne. FAISS FlatL2 ici ; avec vecteurs normalisés, les deux classent pareil.
Chunk size / overlap : trade-offs ?	Gros chunks = plus de contexte mais bruit + tokens ; petits = précis mais fragmentés. Overlap 200 préserve les frontières. On tune par eval (fidélité des citations).
Top-k trop grand/petit ?	Trop petit = réponse incomplète ; trop grand = bruit + coût tokens + dilution de l'attention (« lost in the middle »).
Comment gérez-vous les figures/UML ?	Captions détectées (regex EN/FR) + compteur d'images par page en métadonnées ; le prompt ordonne de nommer et détailler chaque figure. Limite honnête : pas de vision — le modèle ne « voit » pas l'image.
Isolation multi-tenant ?	Un index FAISS par <code>user_id</code> + filtre <code>user_id</code> à la requête : aucun chunk d'un autre user ne peut remonter.
Échecs typiques du RAG ?	Mauvais chunking, k mal réglé, requête vague ($\rightarrow$ reformulation), contexte contradictoire ($\rightarrow$ instruction de priorité), figures purement visuelles.

17. LLM & Groq approfondis (niveau entretien)

17.1 Fondamentaux à maîtriser

- **Tokens** : unités de texte (~4 caractères/token en anglais). Tout se facture/compte en tokens : prompt + raisonnement + réponse.
- **Fenêtre de contexte** : 131k tokens ici — borne la taille historique + RAG injectable (d'où le cap 50 messages et 1800 car./bloc).
- **Modèles de raisonnement** (GPT-OSS) : ils « réfléchissent » en tokens *avant* de répondre → d'où `GROQ_MAX_TOKENS=4096` (1024 tronquait). Le fallback ne se déclenche que sur `model_not_found` , pas sur les 429.
- **Rôles** : `system` (règles, non négociable), `user` , `assistant` . Notre system = règles maison + contexte RAG (+ instruction Agent).
- **Groq/LPU** : puces d'inférence déterministes ultra-rapides (vs GPU généralistes) → latence sub-seconde, quotas stricts (429).
- **Hallucinations** : mitigées par RAG + citations exigées + sanitize (jamais de bulle d'erreur renvoyée au modèle).
- **AsyncGroq partagé** : 1000 chats concurrents multiplexés sur l'event loop au lieu de 40 threads (leçon du test de charge).

17.2 Questions d'entretien LLM (avec réponses modèles)

Question	Bonne réponse (courte)
Temperature / top-p, ça change quoi ?	Contrôlent l'aléatoire : bas (≈0) = factuel/déterministe (idéal RAG), haut = créatif. Nous : défaut provider ; piste = baisser pour le factuel.
Pourquoi un modèle « décommissionné » a tout cassé ?	ID en dur → 404 <code>model_not_found</code> → 500. Leçon : modèle configurable + fallback + 502 avec cause, jamais d'ID en dur.
Comment limitez-vous les coûts/latence ?	Historique capé, blocs capés, top-k petit, quota via rate-limit (Redis prévu), infra-test sans IA, quota Groq monitoré (429).
Sync vs async pour les appels LLM ?	Sync bloque un thread (~40 dispos) ; async multiplexe des milliers d'appels : indispensable sous charge.
Prompt injection — vous gérez ?	Partiellement : sanitize (vide/placeholders), rôles validés, RAG prioritaire par instruction. Reste : garde-fous (Llama Guard prévu).
Évaluer la qualité des réponses ?	Aujourd'hui : citations vérifiables + tests. Prévu : votes 👍👎 , fidélité (faithfulness) vs sources, jeux d'éval.

18. Fiche d'entretien express (Q/R tous sujets)

Question piège	Réponse en 2 phrases
500 sur <code>/ask</code> au début ?	Modèle en dur décommissionné → 404 provider. Fix : modèle configurable + fallback + 502 avec <code>detail</code> .
Historique « empoisonné » ?	La bulle d'erreur était renvoyée au modèle. Fix : bulles UI-only, strip avant envoi/persistance, des deux côtés.
Flood <code>store_user</code> au login ?	Objet profil recréé à chaque render → effect relancé. Fix : dépendances primitives + <code>useMemo</code> → 1×1 requêtes.
Crash hydration Next.js ?	<code>localStorage</code> lu au premier render ≠ SSR. Fix : defaults au premier render, adoption post-hydratation + script de boot.
Worker mort à 1000 users (<code>select()</code>) ?	Cap Windows 512 FD/processus. Fix : workers + <code>limit-concurrency</code> (503 honnêtes) + pool Mongo 50 ; prod = Linux.
Pourquoi pas de middleware Clerk ?	Il ne protégeait rien (gardes client) et coûtait un passage Edge par requête. Auth code-splittée par groupes de routes.
Clerk alourdit la landing ?	Provider déplacé dans le groupe dashboard : landing 108 kB, zéro JS d'auth, page serveur statique.
CI verte, c'est quoi ?	pytest 30 + Vitest 26 + Playwright + <code>next build</code> + Sonar (service éphémère) + Locust 10k en artefacts.
PDF + question ensemble, comment ?	Chip d'attente → upload XHR 0–100 % au send → <code>query_rag</code> immédiat → <code>/ask</code> avec contexte.
Modèle commutable, garde-fous ?	Allow-list <code>/api/models</code> , 400 sinon, fallback auto, modèle utilisé retourné et affiché.

Conseil dernier jour : pour chaque réponse, cite le fichier (`ai_service.py`, `useChat.ts` ...), le chiffre (50 messages, `top_k=4`, 4096 tokens, 512 FD) et l'incident vécu — c'est ce qui transforme un entretien théorique en preuve d'expérience.